University of Canberra
Faculty of Science and Technology

Software Technology 2
(7170)
Week 5 Tutorial & Computer Lab
Assignment 1: Individual Project
Part 2: Recursion and Merge-Sort Algorithms

- This lab activity is assessed and contributes to the Assignment 1 (Individual Project) mark.
- Please go through Week 5 lecture recordings and Week_5_Tutorial_and Computer_Laboratory.doc available after Module 5 in Canvas (Assignment 1/Individual Project preparation activities)
- For assessment of this activity, you need to complete the following in week 5 tutorial/lab:

  1) attend your allocated computer lab class in Week 5,

  2) complete all tasks listed in this handout,

  3) demonstrate the working activities to your tutor in class, and

  4) Save your python project to a zip file and upload this zip file to **In-class Lab Work Demo Week 5** and upload it to your GitHub Repository. Make sure you include a commit message with date and time, when you upload this zip file in GitHub.
- Your mark is 0 if you do not demonstrate your project.
- You can leave the class after you complete the 4 requirements above.

Tasks

- Understanding Recursion Algorithm in Python (recursion_mergesort.py in Week 5 lab activities folder)
- Understanding an sorting algorithm – Merge Sort Algorithm and benchmarking the performance of Merge Sort with Insertion Sort algorithm
- Understanding Tower of Hanoi problem using recursion.
- Solving Fractal problems using recursion

Download the Week_5_Lab_Activities.zip after Module 5 in Canvas, and unzip the folder. This folder content contains all the necessary activities needed for each task completion in this week.

Task 1:Understanding Recursion algorithm and testing it for simple and complex cases (Fibonacci sequence and factorial computation)

In this task, you will compare the recursion with iteration approach for doing repeated execution of task.

(1) Extract the relevant recursion sections from recursion_mergesort.py to understand how simple recursion works for different test cases - simple recursion, Fibonacci sequence and the Factorial computation test cases.
(2) Convert the python code for the three test cases in (1) to iteration (iterative loop) implementation instead of recursion.
(3) Compare iterative vs. recursive implementation for three test cases above for different data sizes.

Report on the comparison results with individual python scripts and screen shots of the outcomes.

Task 2: Understanding an efficient sorting algorithm – Merge Sort Algorithm

In this task, you will learn about an efficient sortin algorithm called the Merge Sort Algorithm for working with simple data structures such as lists and dictionaries.

Open the **recursion_mergesort.py** from the unzipped folder, and extract the relevant section for merge sort algorithm and complete the following steps:

(1) Test the merge-sort algorithm for different data sizes

(2) Test an inefficient sort, such as insertion sort algorithm with similar test data sequences as in (1)

(3) Benchmark the algorithm efficiency (comparison of merge-sort with insertion sort) with TimeIt module

Show the individual script results for each of the above steps and comment on which algorithm performs better and why ?

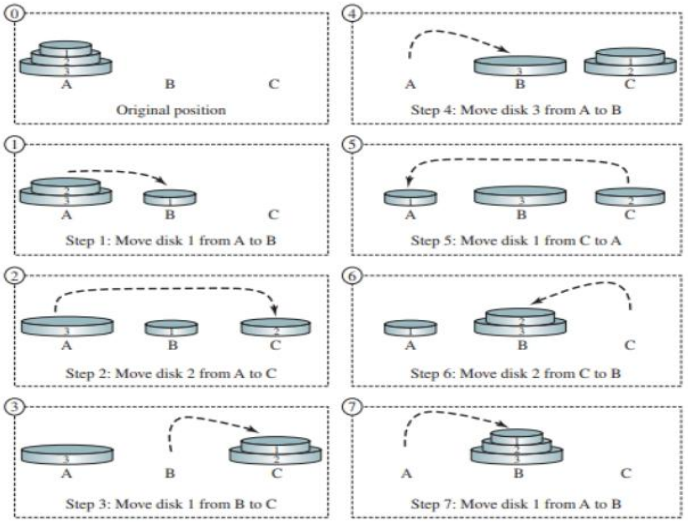
Task 3: Tower of Hanoi problem – Application of recursion algorithm

The Tower of Hanoi problem is a classic problem that can be solved easily by using recursion, but it is difficult to solve otherwise.

The problem involves moving a specified number of disks of distinct sizes from one tower to another while observing the following rules:

- There are n disks labeled 1, 2, 3, ..., n, and three towers labeled A, B, and C.
- No disk can be on top of a smaller disk at any time.
- All the disks are initially placed on tower A.
- Only one disk can be moved at a time, and it must be the top disk on a tower.

The objective of the problem is to move all the disks from tower A to tower B with the assistance of tower C. For example, if you have three disks, the steps to move all of the disks from tower A to B are shown in Figure below:



In this task, you need to do the following:

1. Test the following python script for tower of hanoi problem for different disk sizes and report the results for atleast 2 different disk sizes.

```
# TowerOfHanoi.py
def main():
    n = eval(input("Enter number of disks: "))

    # Find the solution recursively
    print("The moves are:")
    moveDisks(n, 'A', 'B', 'C')

# The function for finding the solution to move n disks
#   from fromTower to toTower with auxTower
def moveDisks(n, fromTower, toTower, auxTower):
    if n == 1: # Stopping condition
        print("Move disk", n, "from", fromTower, "to", toTower)
    else:
        moveDisks(n - 1, fromTower, auxTower, toTower)
        print("Move disk", n, "from", fromTower, "to", toTower)
        moveDisks(n - 1, auxTower, toTower, fromTower)

main() # Call the main function
```

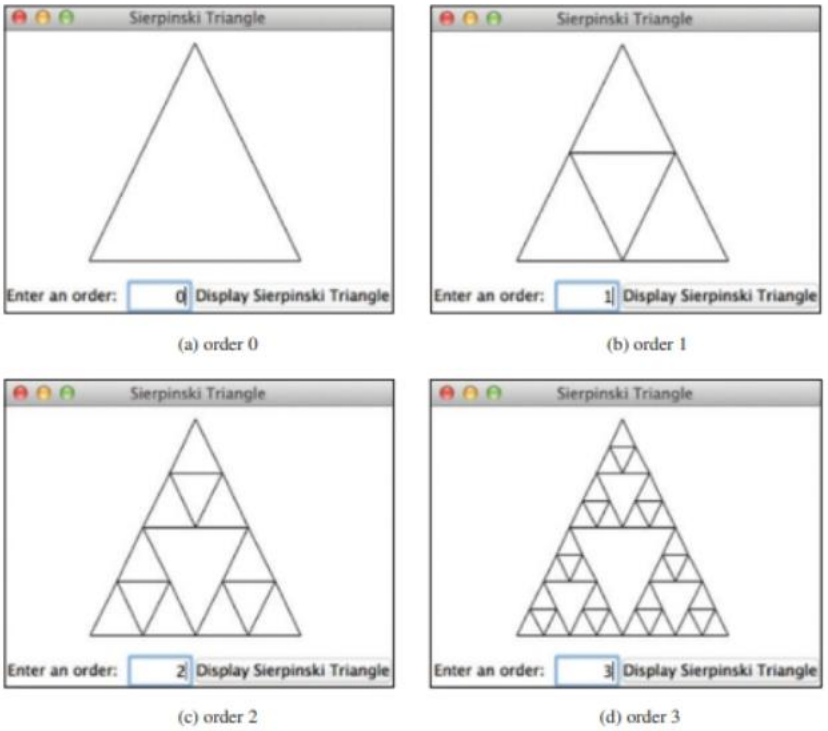
2. Modify the code for TowerOfHanoi.py, so that the program finds the number of moves needed to move n disks from tower A to tower B. (Hint: Use a global variable and increment it for every move.). Test the code for two different disk sizes and report the results. A successful test run should display the following at the console.

```
Enter number of disks: 8
The moves are:
Number of moves is 255
```

Task 4: Sierpinski Triange – Fractals Computation using recursion algorithm

Fractals Recursion is ideal for displaying fractals, because fractals are inherently recursive.
A fractal is a geometrical figure, but unlike triangles, circles, and rectangles, fractals can be divided into parts, each of which is a reduced-size copy of the whole. There are many interesting examples of fractals. One of the most simple fractal - the Sierpinski triangle, named after a famous Polish mathematician. A Sierpinski triangle is created as follows:

1. Begin with an equilateral triangle, which is considered to be a Sierpinski fractal of order (or level) 0, as shown in Figure a below.
2. Connect the midpoints of the sides of the triangle of order 0 to create a Sierpinski triangle of order 1 (Figure b).
3. Leave the center triangle intact. Connect the midpoints of the sides of the three other triangles to create a Sierpinski triangle of order 2 (Figure c).
4. You can repeat the same process recursively to create a Sierpinski triangle of order 3, 4, and so on.



(a) order 0 (b) order 1 (c) order 2 (d) order 3

(1) You need to run the TkInter code sierpinski_triangle.py for this simple fractal problem and test it for atleast 3 different orders other than 0,1,2,and 3, and report the results(screen shots).

```
#sierpinski_triangle.py
from tkinter import * # Import tkinter

class SierpinskiTriangle:
    def __init__(self):
        window = Tk() # Create a window
        window.title("Sierpinski Triangle") # Set a title

        self.width = 200
        self.height = 200
```

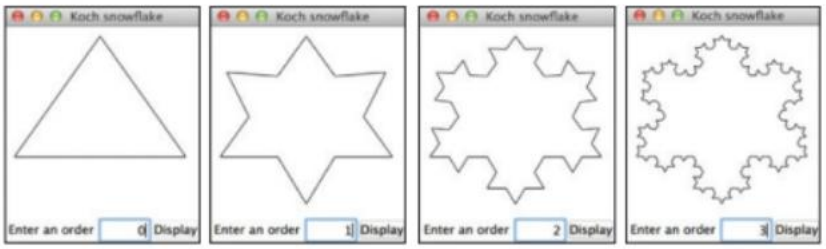
```
        self.canvas = Canvas(window,
            width = self.width, height = self.height)
        self.canvas.pack()

        # Add a label, an entry, and a button to frame1
        frame1 = Frame(window) # Create and add a frame to window
        frame1.pack()

        Label(frame1,
            text = "Enter an order: ").pack(side = LEFT)
        self.order = StringVar()
        entry = Entry(frame1, textvariable = self.order,
                      justify = RIGHT).pack(side = LEFT)
        Button(frame1, text = "Display Sierpinski Triangle",
            command = self.display).pack(side = LEFT)

        window.mainloop() # Create an event loop

    def display(self):
        self.canvas.delete("line")
        p1 = [self.width / 2, 10]
        p2 = [10, self.height - 10]
        p3 = [self.width - 10, self.height - 10]
        self.displayTriangles(int(self.order.get()), p1, p2, p3)

    def displayTriangles(self, order, p1, p2, p3):
        if order == 0: # Base condition
            # Draw a triangle to connect three points
            self.drawLine(p1, p2)
            self.drawLine(p2, p3)
            self.drawLine(p3, p1)
        else:
            # Get the midpoint of each triangle's edge
            p12 = self.midpoint(p1, p2)
            p23 = self.midpoint(p2, p3)
            p31 = self.midpoint(p3, p1)

            # Recursively display three triangles
            self.displayTriangles(order - 1, p1, p12, p31)
            self.displayTriangles(order - 1, p12, p2, p23)
            self.displayTriangles(order - 1, p31, p23, p3)

    def drawLine(self, p1, p2):
        self.canvas.create_line(
            p1[0], p1[1], p2[0], p2[1], tags = "line")

    # Return the midpoint between two points
    def midpoint(self, p1, p2):
        p = 2 * [0]
        p[0] = (p1[0] + p2[0]) / 2
        p[1] = (p1[1] + p2[1]) / 2
        return p

SierpinskiTriangle() # Create GUI
```

(2) Now, you will modify the sierpinski_triangle.py program to display snowflake fractal, another fractal named after Koch a Swedish mathematician. fractal, called the Koch snowflake, named after a famous Swedish mathematician.

Provide the code **koch_snowflake.py**, and test results for the code you have written, and report the results for atleast 3 different orders (include the screenshots).

Note: A Koch snowflake is created as follows:

a. Begin with an equilateral triangle, which is considered to be the Koch fractal of order (or level) 0, as shown in Figure a.
b. Divide each line in the shape into three equal line segments and draw an out-ward equilateral triangle with the middle line segment as the base to create a Koch fractal of order 1, as shown in Figure b.
c. Repeat Step 2 to create a Koch fractal of order 2, 3, ..., and so on, as shown in Figure c–d



(a) (b) (c) (d)

- Please complete this report with your python scripts and test results, and your comments for each task in relation to the questions asked.
- Demonstrate/present the tasks to your tutor in class.
- Prepare a report, similar to this handout, but with code/scripts and results included for your tests for each task, and save the report as **Week_5_lab_report.pdf**.
- Include this report as well as python scripts for each task (task 1 to task 4) in a separate folder and zip it, with a title (**Week_5_LabAcitivities.zip**).
- Upload the zip file to your github repo, and add a commit message with date and time details.
- You can leave the lab now. Please note your mark is 0 if you do not present/demonstrate your working project in lab class. This must be done in **Week 5**.

Mark for this tutorial: 8 marks